

Laborator 8

Implementarea sistemelor în timp real Comutarea între task-uri cu durată infinită

Obiectiv În acest laborator se va prezenta modalitatea în care se efectuează comutarea între task-uri cu durată infinită, așa cum sunt folosite ele în majoritatea sistemelor în timp real. Metoda se va prezenta comparativ cu celelalte metode de comutare utilizate în laborator până acum.

Breviar teoretic

În cele prezentate până acum s-au utilizat mai multe metode prin care se putea descrie prin task-uri un sistem în timp real. În funcție de tipul task-ului trebuia aleasă și metoda de comutare între task-uri.

Există trei modalități prin care se pot descrie task-urile:

Metoda 1: Task-uri ca funcții cu durată finită apelate de către administratorul de task-uri

Task-urile sunt funcții cu durată finită, care sunt apelate de către administratorul de task-uri (din subrutina timer).

Descriere

Practic programul rulează în subrutina de tratare a întreruperii de la timer cea care pune în execuție task manager-ul. Comutarea între funcții se realizează prin apelul funcțiilor. După ce funcția este executată, resursele sunt disponibile pentru alte activități până la apariția următoarei subrutine.

Exemplu descriere în cod C:

```
//descriere task-uri
void task1(void)
{
    contor1++;
}

void task2(void)
{
    if(contor2!=contor1)
    {
        contor2=contor1;
        printf("Contor incrementat de %d ori",contor2);
    }
}

//descriere administrator task-uri
void Timer0_ISR (void) interrupt 1
```

```

{
TH0 = 0xf6;//divizare cu 250 - comunicare 9600bps
TL0 = 0x08;//divizare pentru a obtine 100 herti (10 ms)

//incrementare task curent
task_crt++;
if(task_crt==2)
    task_crt=0;

//selectie functie care va fi apelata
switch(task_crt)
{
    case 0: task1();//apel functie task 1
        break;
    case 1: task2();//apel functie task 2
        break;
}
}

```

Avantaje și dezavantaje

Metoda este cea mai simplă de descris. Permite implementarea unui mecanism bazat pe priorități la lansarea în execuție a unui task care poate fi schimbat dinamic. Această metodă nu poate fi utilizată pentru funcții cu durată de execuție mare sau care procesează bucle infinite. Nu permite suspendarea unui task curent în favoarea altuia care ar fi executat, deci nu permite implementarea priorităților cu suspendarea execuției. Nu permite alocarea timpilor diferiți de execuție.

Metoda 2: Task-uri ca bucle cu durată infinită comutate de către administratorul de task-uri

Task-urile sunt bucle cu durată infinită, între care se poate face comutarea de către administratorul de task-uri (din subrutina timer).

Descriere

În această metodă se rulează o funcție task care conține două bucle de această dată cu durată ipotetic nelimitată. De fapt, durata este controlată de administratorul de task-uri, prin variabila task_crt. Programul rămâne blocat într-o buclă până când administratorul decide ieșirea din ea. Administratorul, deși simplificat, are un control mai ridicat asupra task-ului decât în metoda anterioară.

Exemplu descriere în cod C:

```

//descriere task-uri

```

```

void task(void)
{
    for(;;)
    {
        while(task_crt==0)
        {
            contor1++;
        }
        while(task_crt==1)
        {
            if(contor2!=contor1)
            {
                contor2=contor1;
                printf("Contor incrementat de %d ori",contor2);
            }
        }
    }
}

//descriere administrator task-uri
void Timer0_ISR (void) interrupt 1
{
    TH0 = 0xf6;//divizare cu 250 - comunicare 9600bps
    TLO = 0x08;//divizare pentru a obtine 100 herti (10 ms)

    //incrementare task curent
    task_crt++;
    if(task_crt==2)
        task_crt=0;
}

```

Avantaje și dezavantaje

Metoda permite alocarea timpilor de execuție diferiți. Permite implementarea unui mecanism bazat pe priorități la lansarea în execuție a unui task care poate fi schimbat dinamic. Metoda permite suspendarea dar nu pentru funcțiile care sunt apelate în interiorul task-ului. Practic suspendarea înseamnă de fapt terminarea execuției unei parcurgeri de buclă și abia apoi trece la următoarea buclă. De aceea nici această metodă nu este foarte eficientă în cazul unor funcții cu durată de execuție mare apelate în interiorul unui task. Orice buclă introdusă ar trebui să conțină o condiție de suspendare la comutarea între task-uri.

Metoda 3 Task-uri ca funcții cu durată infinită comutate de către administratorul de task-uri

Task-urile sunt funcții cu durată infinită, comutarea între ele este efectuată de către administrator.

Descriere

Spre deosebire de celelalte metode, aceasta necesită anumite cunoștințe despre procesorul pe care se realizează aplicația. De asemenea sunt necesare noțiuni legate de mecanismul de apel al unei funcții, noțiuni prezentate la materiile „Sisteme cu microprocesoare” și „Microprocesoare” rezumate în cele ce urmează.

Astfel, orice procesor posedă un registru care are stocată adresa instrucțiunii imediat următoare celei curente, registru numit numărător de program sau Program Counter (PC). Înainte de a trece la executarea instrucțiunii, procesorul consultă registrul PC, află adresa instrucțiunii care trebuie să o execute și apoi citește de la această adresă codul instrucțiunii pe care îl execută. Registrul PC este reactualizat la fiecare instrucțiune executată cu adresa instrucțiunii următoare. Pornind de la acest mecanism, utilitatea registrului PC este extinsă. Astfel, reset-ul aplicat unui procesor semnifică de fapt încărcarea în PC a adresei 0. O instrucțiune de salt constă de fapt cu încărcarea în PC a adresei de salt.

În cazul în care în program este apelată o funcție, conținutul registrului PC este salvat în stivă, este încărcat cu adresa primei instrucțiuni din funcție, se execută întreaga funcție apoi este restaurat din stivă cu adresa instrucțiunii care urmează funcției ca în exemplul de mai jos:

```
int suma(int a, int b)
{
    return a+b;
}
```

```
void main(void)
{
    int a,b;
    a = 10;
    b = 20;
    c = suma(a,b);
    printf(“%d”,c);
}
```

Operatii efectuate:

Adresă	Instrucțiune	PC	Stivă	Descriere
0	a=10;	1		Execută instrucțiunea de la adresa 0. PC conține adresa următoarei instrucțiuni adică 1
1	b=20;	2		Execută instrucțiunea de la adresa 1
2	suma(a,b);	3		Execută apelul funcției „suma”. PC conține adresa următoarei instrucțiuni care este 3

	apel suma Suma→PC→stiva	Suma	3	PC este salvat în stivă iar adresa funcției „sumă” este încărcată în PC
Suma	a+b;	Suma+1	3	Execută instrucțiunea din funcție. PC este incrementat cu adresa următoare
	return suma stiva → PC	3		La ieșirea din funcție PC se restaurează cu adresa din stivă.
3	Printf	4		Execută instrucțiunea următoare

Pentru accesul la stivă există un registru specializat, denumit SP, care indică vârful stivei. La C8051F120, SP se incrementează la fiecare depunere în stivă și se decrementează când se extrage din stivă. La depunere în stivă secvența este următoarea:

1. incrementează registrul SP,
2. depune în stivă.

La extragerea din stivă secvența este:

1. extrage din stivă,
2. decrementează SP.

Alături de registrul PC, în stivă este depus și conținutul unor regiștrii care sunt afectați de subrutină.

O procedură similară are loc și la executarea unei subrutine de tratare a întreruperii. Practic, subrutina de tratare a întreruperii este organizată logic ca mai jos:

PUSH PC	;salveaza PC in stiva
PUSH PSW	;salveaza registru indicatori in stiva
.....	;salveaza alti registrii utilizati in subrutina
	;corpul subrutinei
.....	;restaureaza alti registrii utilizati in subrutina
POP PSW	;restaureaza registru indicatori
POP PC	;restaureaza PC

De exemplu, o subrutină de tratare a întreruperii de la canalul timer care arată descrisă în C așa:

```
//subrutina tratare timer
void Timer0_ISR (void) interrupt 1
{
    TH0 = 0xf6;//divizare cu 250 - comunicare 9600bps
    TL0 = 0x08;//divizare pentru a obtine 100 herti (10 ms)

    if(cont_tasta)
        cont_tasta--;
    .....
}
```

Arată dezasamblată astfel:

```
B0:04D5 C0 E0      PUSH E0H
B0:04D7 C0 D0      PUSH D0H
B0:04D9 75 8C F6    MOV  8CH,#F6H
B0:04DC 75 8A 08    MOV  8AH,#08H
B0:04DF E5 6A      MOV  A,6AH
B0:04E1 60 02      JZ    02H
.....
B0:0505 D0 D0      POP   D0H
B0:0507 D0 E0      POP   E0H
B0:0509 32         RETI
```

Se observă ca începe cu mai multe instrucțiuni PUSH (depunere stivă) și se termină cu mai multe instrucțiuni POP (extragere stivă). **Vizualizarea programelor în dezasamblare se face în mediul SiLabs IDE implicit, la rularea programului sau prin acționarea butonului Disassembly Windows din bara cu butoane.**

Bazându-se pe acest mecanism, tehnica de comutare între două task-uri descrise ca două funcții consta în comutarea stivei alocate pentru fiecare din cele două task-uri. Astfel, fiecare task are propria lui stivă, descrisă ca o zonă de memorie RAM în care se vor salva PC, PSW și alți regiștrii. Operațiile într-o subrutină vor fi în acest caz: salvare în stivă task i, comutare task i+1 și restaurare din stiva task i+1:

PUSH PC	;salveaza PC in stiva task i
PUSH PSW	;salveaza registru indicatori in stiva task i
.....	;salveaza alti registrii utilizati in stiva task i
	;corpul subrutinei
SP = stiva_taski+1;	;comutare task urmator
.....	;restaureaza alti registrii utilizati in subrutina stiva taski+1
POP PSW	;restaureaza registru indicatori stiva taski+1
POP PC	;restaureaza PC stiva taski+1

Salvarea registrului PC precum și a celorlalți regiștrii este efectuată automat de către compilator. Aceasta generează o problemă la prima rulare a task-urilor. Sistemul este demarat cu apelul din programul principal a primului task care se dorește să se execute. Însă, task-ul 2 și următoarele după el au o problemă în sensul că prima oară când sunt trecute în starea curentă se efectuează o operație de preluare din stiva lor înainte de a fi depus ceva acolo! De aceea, inițializarea acestora se face prin scrierea manuală în stiva fiecăruia a adresei funcției task și a numărului de regiștrii pe care compilatorul a decis să îi salveze în stivă (acest număr poate fi determinat rulând programul și observând în dezasambelor câți regiștrii sunt salvați de compilator).

Exemplu descriere în cod C:

```
//descriere task-uri
void task1(void)
{
    for(;;)
    {
        contor1++;
    }
}

void task2(void)
{
    for(;;)
    {
        if(contor2!=contor1)
        {
            contor2=contor1;
            printf("Contor incrementat de %d ori",contor2);
        }
    }
}

//subrutina tratare timer
void Timer0_ISR (void) interrupt 1
{
    TH0 = 0xf6;//divizare cu 250 - comunicare 9600bps
    TL0 = 0x08;//divizare pentru a obtine 100 herti (10 ms)

    //comutare task-uri
    task_crt++;
    if(task_crt==2)
        task_crt=0;
    switch(task_crt)
    {
        case 0:
            task2.pstiva=SP;
            SP = task1.pstiva;
            break;
        case 1:
            task1.pstiva=SP;
            SP = task2.pstiva;
            break;
    }
}
```

}

Avantaje și dezavantaje

Metoda este cel mai dificil de descris deoarece solicită cunoștințe suplimentare despre procesor și despre compilator. Dar metoda permite implementarea unui mecanism bazat pe priorități cu suspendarea execuției și permite alocarea timpilor de execuție diferiți pentru fiecare task. Nu apar nici un fel de probleme și nici un fel de modificări suplimentare dacă se execută funcții cu durată de timp ridicată sau chiar bucle infinite în interiorul unui proces. De asemenea oferă cel mai rapid răspuns la comutare (la celelalte metode la comutare se efectuau operații suplimentare înainte de efectuarea comutării). Aceasta este cea mai optimă metodă folosită la un sistem în timp real care lucrează cu funcții task cu durată mai mare de timp.

Desfășurarea lucrării

Primele două metode au fost exemplificate în laboratoarele anterioare. În cele ce urmează se va prezenta o problemă care va trata cea de-a treia metodă.

Problema realizează două task-uri: un task care se ocupă de tratarea unei taste iar cel de-al doilea de comunicarea serială a numărului de apăsări de taste. Comutarea între cele două task-uri se bazează pe un mecanism de ordine, la un interval fix de 10ms.

Problema la apăsarea unei taste este aceea că, pentru a elimina efectul de oscilații care apar la orice contact mecanic, trebuie lăsată o temporizare. Această temporizare însă poate fi nedorită dacă se folosesc celelalte două metode. Cu cea de-a treia metodă descrierea se face în cel mai simplu mod cu puțință așa cum se vede mai jos:

```
//task 1 folosit pentru citirea tastei
void task1(void)
{
    for(;;)
    {
        if(TASTA==0)//tasta apasata ?
        {
            cont_tasta=1;//daca da pregateste contorul temporizator
            while(cont_tasta);//asteapta temporizarea
            if(TASTA==0)//tasta in continuare apasata?
            {
                contor1++;//daca da incrementeaza contorul tastei
                while(!TASTA);//asteapta eliberarea tastei
            }
        }
    }
}
```

```

}
}
}

```

Cont_tasta se decrementează în subrutina Timer care apare la 10 ms.
Cel de-al doilea task, care se ocupă cu transmisia serială, arată în felul următor:

```

//task 2 : transmite serial
void task2(void)
{
    for(;;)
    {
        if(contor1!=contor2)//daca numarul de apasari s-a modificat
        {
            contor2=contor1;//reactualizeaza
            printf("S-a apasat tasta a %x oara\n",contor2);//transmite
        }
    }
}

```

Codul sursă complet al programului este descris mai jos:

```

#include <c8051f120.h> //fisier pentru declaratii
#include <stdio.h>

//declaratii LED
sbit LED = P1^6;
sbit TASTA = P3^7;
#define SYSCLK 3062500

//variabile globale folosite in program

unsigned char task_crt=0;//task curent

unsigned int point;//pentru conversie pointer functie

typedef struct Task_ram
{
    unsigned char stiva_hard[32];//stiva hardware
    //unsigned char stiva_soft[8];//stiva software - set de variabile folosit
    unsigned char pstiva;//registru SP
}task_ram;

```

```

//variabile globale salvate la comutarea task-uri
unsigned int contor1=0;
unsigned int contor2=0;
unsigned char cont_tasta=0;

//declaratie numar de task-uri
task_ram task[2];

//task 1 folosit pentru citirea tastei
void task1(void)
{
    for(;;)
    {
        if(TASTA==0)//tasta apasata ?
        {
            cont_tasta=1;//daca da pregateste contorul temporizator
            while(cont_tasta);//asteapta temporizarea
            if(TASTA==0)//tasta in continuare apasata?
            {
                contor1++;//daca da incrementeaza contorul tastei
                while(!TASTA);//asteapta eliberarea tastei
            }
        }
    }
}

//task 2 : transmite serial
void task2(void)
{
    for(;;)
    {
        if(contor1!=contor2)//daca numarul de apasari s-a modificat
        {
            contor2=contor1;//reactualizeaza
            printf("S-a apasat tasta a %d oara\n",contor2);//transmite
        }
    }
}

//functii initializare
//initializare sistem
void Init_System(void)
{
    //dezactivare timer watch-dog

```

```
WDTCN = 0xde; //succesiune de secvente pentru dezactivare watchdog
WDTCN = 0xad;
}
```

```
//initializare canale timer 0 si 1
void Init_TMR(void)
{
    SFRPAGE = TIMER01_PAGE; //pregateste pagina pentru configurare timer 0 si 1
    TCON = 0x50; //activare canal timer 0 si timer 1
    TMOD = 0x21; //temporizator canal 1 mod 2 (8 biti cu revenire), canal 0 mod 1 (16 biti fara revenire)
    CKCON = 0x10; //selectie sursa ceas timer 1 - fara prescaler 12
    TH1 = 0xfb; // - comunicare 19200bps
    TH0 = 0x9c; //divizare pentru a obtine 100 herti (10 ms) la timer 0
    TL0 = 0x50; //
}
```

```
//initializare porturi I/O
void Init_PORT(void)
{
    SFRPAGE = CONFIG_PAGE; //pagina pentru configurare porturi
    P0MDOUT = 0x03; //configureaza pinul TX ca iesire
    XBR0 = 0x04; //configureaza portul UART0 pe porturile 0
    //configurare port 2
    XBR2 = 0x40; //configureaza liniile de port
    P1MDOUT |= 0x40; //configureaza pinul P1.6 ca iesire (LED)
}
```

```
//initializare canal serial
void Init_UART0(void)
{
    SFRPAGE = UART0_PAGE; //setare pagina SFR pentru canalul 0 UART
    SCON0 = 0x50; //serial modul 1 receptie activata
    TI0 = 1; //initializare transmisie seriala
}
```

```
//activare intreruperi
void enable_interrupt(void)
{
    SFRPAGE = LEGACY_PAGE;
    //activare intrerupere
    IE = 0x92;
    //initializare pagina seriala
    SFRPAGE = UART0_PAGE;
}
```

```

void main(void)
{
    Init_System();//initializare sistem
    Init_TMR();//initializare canale timer
    Init_PORT();//initializare porturi I/O
    Init_UART0();//initializare canale seriale

    //initializare stiva task 1
    point = (unsigned int) task1;//conversie adresa functie
    task[0].stiva_hard[1]=(unsigned char) ((point>>8)&0x00ff);//salvare adresa stiva
    task[0].stiva_hard[0]=(unsigned char) (point&0x00ff);
    task[0].pstiva= (task[0].stiva_hard)-1;//salvare stiva

    //initializare stiva task 2
    point = (unsigned int) task2;//conversie adresa functie
    task[1].stiva_hard[1]=(unsigned char) ((point>>8)&0x00ff);//salvare adresa stiva
    task[1].stiva_hard[0]=(unsigned char) (point&0x00ff);
    task[1].pstiva= (task[1].stiva_hard)+3;//salvare stiva

    //initializare stiva pentru task 1
    SP = (task[0].stiva_hard)-1;

    enable_interrupt();//activare intreruperi

    //pornire task 1 initial
    task1();
    for(;;);

    //task2();
}

//subrutina tratare timer
void Timer0_ISR (void) interrupt 1
{
    TH0 = 0xf6;//divizare cu 250 - comunicare 9600bps
    TL0 = 0x08;//divizare pentru a obtine 100 herti (10 ms)

    //contor pentru tastatura
    if(cont_tasta)
        cont_tasta--;

    //comutare task-uri

```

```

task_crt++;
if(task_crt==2)
    task_crt=0;

//initializare adresa stiva task curent
switch(task_crt)
{
    case 0:
        task[1].pstiva=SP;
        SP = task[0].pstiva;
        break;
    case 1:
        task[0].pstiva=SP;
        SP = task[1].pstiva;
        break;
}
}

//subrutina tratare intrerupere seriala
void Serial0_ISR (void) interrupt 4
{
    //receptie caracter serial?
    if(RI0)
    {
        //tratare receptie seriala

        RI0=0;
    }
}

```

Aplicații. Probleme propuse

1. Să se compare cele trei metode de comutare analizând avantajele și dezavantajele lor, precum și situațiile în care se pot folosi fiecare;
2. Se va crea un fișier în care se va introduce programul, se va compila, descărca în machetă și se va observa funcționarea sa. Pentru monitorizarea ieșirii se va utiliza aplicația HyperTerminal din Windows (Start – All Programs – Accessories – Communication) și se va realiza o conexiune serială, viteza 19200, 8 biți date, fără paritate, 1 bit stop.
3. Care este dimensiunea ocupată în memoria RAM de un task declarat ca în programul de mai sus?
4. Care este rolul variabilei pstiva declarată în tipul task_ram?

5. De ce este necesară inițializarea stivei task-urilor în programul principal și rularea primei funcții task?
6. Explicați expresiile din programul de mai sus:
 - a. `task[0].pstiva = (task[0].stiva_hard)-1; //salvare stiva`
 - b. `task[1].pstiva = (task[1].stiva_hard)+3; //salvare stiva`De ce -1 și de ce 3?
7. Să se efectueze un program care într-un task preia numere serial, de la PC (transmise cu hiperterminal) iar în celălalt, numai în cazul în care au fost preluate două numere, determină maximum dintre ele și îl afișează (retransmite serial).